RMA TRACKER DOCUMENTATION LIBRARY

VERSION 2.1.1

VOLUME I

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

RMA Tracker

Defines the functional and nonfunctional requirements, interfaces, constraints, business rules, and acceptance criteria for RMA Tracker Version 2.1.1.

Prepared for: Harp Enterprises

Prepared by: Brennen Gabriel

Document version: 2.1.1

Publication date: July 2026

Table of Contents

1. Introduction

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for **RMA Tracker Version 2.1.1**, developed by Brennen Gabriel for **Harp Enterprises**.

RMA Tracker is a standalone desktop application designed to simplify the management of Return Merchandise Authorization (RMA) processes for organizations responsible for tracking equipment sent for repair, warranty service, refurbishment, or replacement.

This document establishes a common understanding of the system's requirements and expected behavior among developers, testers, maintainers, project stakeholders, and future contributors. It serves as the primary reference throughout the software development life cycle (SDLC), providing a foundation for future maintenance, enhancements, testing, and version control.

1.1. Purpose

The objectives of this specification are to:

- Define the scope of the software.
- Describe functional capabilities.
- Identify system constraints and assumptions.
- Specify performance and quality requirements.
- Provide a foundation for testing and validation.
- Support future software enhancements and future commercialization.

1.2. Scope

RMA Tracker Version 2.1.1 is a Java desktop application that manages the complete life cycle of Return Merchandise Authorizations (RMAs).

The application enables users to:

- Create and manage RMA records.
- Store customer information.
- Track multiple repair items within a single RMA.
- Record machine information including:
    - Machine ID
    - Serial Number
    - Machine Type
    - Description
    - Problem Description
- Maintain shipping information.
- Record incoming and outgoing tracking numbers.
- Monitor repair status changes.

- Maintain a permanent status history.
- Search and filter RMAs.
- Generate professional reports.
- Print reports.
- Export reports to Microsoft Excel.
- Export reports to PDF.
- Automatically back up the database.
- Restore previous backups.
- Maintain data within a local SQLite database.

Version 2.1.1 focuses on providing a stable desktop solution for internal business use while establishing the architectural foundation for future enterprise capabilities including cloud synchronization, user authentication, dashboard reporting, automated shipping integration, and machine inventory management.

The system is intended for organizations that manage moderate to large volumns of repair records while minimizing paperwork, improving traceability, and reducing administrative overhead.

1.3. Intended Audience

This document is intended for the following audiences and stakeholders.

1.3.1. Developers

Software engineers responsible for implementing new functionality, correcting defects, and maintaining future versions of RMA Tracker.

1.3.2. Test Engineers

Quality assurance personnel responsible for validating software functionality against documented requirements.

1.3.3. System Administrators

Personnel responsible for installing, deploying, backing up, and maintaining the application.

1.3.4. Project Managers

Individuals responsible for project planning, scheduling, documentation, and release management.

1.3.5. Technical Writers

Personnel responsible for creating user manuals, installation guides, and training materials.

1.3.6. Business Stakeholders

Managers and decision makers who require an understanding of the application's capabilities, limitations, and future roadmap.

1.3.7. End Users

Administrative personnel responsible for creating, maintaining, searching, reporting, and tracking RMA records.

1.4. Definitions, Acronyms, and Abbreviations

RMA	**Return Merchandise Authorization**
SQLite	Lightweight embedded relational database used by the application
GUI	Graphical User Interface
PDF	Portable Document Format
XLSX	Microsoft Excel Workbook format
CRUD	Create, Read, Update and Delete operations
SDLC	Software Development Life Cycle
JVM	Java Virtual Machine
JavaFX	Java graphical user interface framework used by the application
Backup	A copy of the database created for recovery purposes
Restore	Process of replacing the current database with a previous backup
Status History	Chronological log of all status changes made to an RMA
Repair Item	Individual machine or device associated with an RMA
Report Preview	On-screen display of a formatted RMA report before printing or exporting
Export	Process of saving application data into another file format such as PDF or Excel

1.5. References

The following references were used during the design and implementation of RMA Tracker Version 2.1.1.

1.5.1. Standards

- IEEE Std 830-1998 – Software Requirements Specification
- IEEE 29148 – Systems and Software Engineering Requirements Engineering

1.5.2. Technologies

- Java Development Kit (JDK 24)
- JavaFX

- SQLite
- Gradle Build System
- Apache POI (Excel Export)
- PDF Export Library
- IntelliJ IDEA

1.5.3. Internal Documentation

- RMA Tracker Version 2.1.1 Source Code
- Volume II – Technical Design & Architecture
- Volume III – User Manual
- Volume IV – Installation, Deployment & Maintenance Guide

1.6. Document Organization

This Software Requirements Specification is organized into the following major sections:

Section	Description
Section 1	Introduction
Section 2	Overall Description
Section 3	Functional Requirements
Section 4	External Interface Requirements
Section 5	Nonfunctional Requirements
Section 6	Data Requirements
Section 7	Business Rules
Section 8	Use Cases
Section 9	Future Enhancements
Section 10	Acceptance Criteria

Section 11	Glossary
Section 12	Appendices

2. Overall Description

RMA Tracker Version 2.1.1 is a standalone desktop application developed by Brennen Gabriel for Harp Enterprises to manage the complete lifecycle of Return Merchandise Authorizations (RMAs). The application centralizes customer records, repair information, shipping details, reporting, and historical tracking within a single integrated system.

The software was created to replace spreadsheet-based, paper-based, and other manual tracking methods with a structured and searchable digital solution. By storing RMA information in a local SQLite database, the application improves data organization, record traceability, reporting consistency, data integrity, and the efficiency of day-to-day repair administration.

Version 2.1.1 focuses on providing a stable, maintainable, and easy-to-use desktop solution for Harp Enterprises. The current release supports the management of RMA records, customer details, multiple repair items, shipping information, status history, searches, filters, reports, printing, Excel exports, PDF exports, and database backup and recovery operations. These capabilities enable users to efficiently create, maintain, search, and archive RMA records while supporting daily repair and warranty workflows.

The application currently operates as a standalone local system. However, its modular architecture provides a foundation for future commercial and enterprise capabilities, including cloud synchronization, user authentication, role-based access control, automated tracking updates, machine inventory management, dashboards, and multi-user collaboration.

The following subsections describe the product perspective, product functions, operating environment, design constraints, assumptions and dependencies, external interfaces, and the expected future evolution of RMA Tracker Version 2.1.1.

2.1. Product Perspective

This section describes how RMA Tracker Version 2.1.1 fits within the operational environment of Harp Enterprises. It provides a high-level description of the software's purpose, its relationship to existing business processes, the major architectural components that make up the application, its interaction with external systems, and the long-term vision for future development. It establishes the architectural context for the functional requirements presented in Section 3.

RMA Tracker was developed by Brennen Gabriel for Harp Enterprises to replace spreadsheet-based and paper-based RMA tracking with a centralized desktop application. By combining customer information, repair records, machine information, shipping details, reporting, and historical tracking into a single application, the software improves operational efficiency, increases traceability, and reduces administrative effort, and provide a single source of truth for all RMA records

The application follows a modular layered architecture that separates the user interface, business logic, reporting services, backup services, and database access. This design improves maintainability, simplifies testing, supports scalability, and provides a stable foundation for future enterprise enhancements.

The following subsections describe the product from an architectural perspective before the individual functional requirements are introduced in Section 3.

2.1.1. System Overview

RMA Tracker Version 2.1.1 is a standalone desktop application that manages the complete lifecycle of Return Merchandise Authorization (RMA) records from creation through final shipment and archival. The application provides a centralized environment for storing customer information, repair items, shipping records, status histories, and reporting information. The application is designed for internal organizational use and requires no network connectivity for core functionality.

Unlike spreadsheet-based tracking systems, RMA Tracker stores all information within a SQLite relational database. This approach improves data integrity, reduces duplicate information, supports rapid searching and filtering, and provides a permanent historical record of repair activity.

Each RMA may contain multiple repair items, allowing organizations to track several machines under a single authorization while maintaining complete repair histories.

2.1.2. Major Software Components

The application is divided into logical software components that isolate responsibilities and simplify maintenance.

User Interface Layer
• JavaFX desktop interface
• Main window, dialogs, reports, menus, and search tools

Business Logic Layer
• Validation
• Business rules
• Search processing
• Status management
• Report generation
• Backup coordination

Data Access Layer
• CRUD operations

• SQL execution
• Transaction management
• Database integrity verification

Reporting Subsystem
• Print Preview
• PDF generation
• Excel export
• Printable reports

Backup and Recovery Subsystem
• Automatic backups
• Manual backups
• Database restore
• Backup validation

2.1.3. System Architecture

RMA Tracker follows a layered architecture in which presentation, business logic, data access, and persistent storage are separated into independent layers. This design reduces coupling, improves maintainability, and allows future enhancements without extensive redesign.

Architecture Overview

User Interface (JavaFX)
↓
Business Logic Layer
↓
Data Access Layer
↓
SQLite Database

2.1.4. External Interfaces

Version 2.1.1 interfaces with the Java Runtime Environment, JavaFX Runtime, SQLite Database Engine, Apache POI for Microsoft Excel generation, PDF generation libraries, the local file system, printers, and standard desktop input devices. All external dependencies are locally installed and do not require Internet connectivity for normal operation.

Future versions are expected to integrate with shipping carrier APIs (UPS, FedEx, USPS), cloud-hosted databases, REST services, barcode scanners, and customer web portals.

2.1.5. Future Product Evolution

The architecture implemented in Version 2.1.1 was intentionally designed to support future commercial growth. Planned enhancements include cloud synchronization, multi-user access, user authentication, role-based security, machine inventory management, automated shipment tracking, dashboard analytics, REST API integration, mobile device support, email notifications, and a customer self-service portal.

These enhancements will allow RMA Tracker to evolve from a standalone desktop application into a scalable enterprise repair management platform while preserving compatibility with the existing data model and minimizing migration effort for existing customers.

2.2. Product Functions

This section provides a high-level overview of the primary functional capabilities offered by RMA Tracker Version 2.1.1. Rather than describing the detailed software requirements, this section summarizes the major business functions available to users and explains how each function contributes to the overall repair management process. It provides a functional overview of the system prior to the detailed requirements defined in Section 3

RMA Tracker was developed by Brennen Gabriel for Harp Enterprises to simplify the management of Return Merchandise Authorizations by integrating customer records, repair information, shipping details, reporting, and historical tracking into a single desktop application. Collectively, these functions provide a complete workflow for managing equipment from the time an RMA is created until the repair is completed and the equipment is returned to the customer. The integrated workflow improves operational efficiency, data consistency, and repair traceability.

The following subsections summarize the major functional areas of the application. Detailed functional requirements for each capability are presented later in Section 3 of this Software Requirements Specification.

2.2.1. RMA Management

RMA management is the core function of the application. Users can create, edit, view, and delete RMA records while maintaining customer information and monitoring the overall repair lifecycle. Each RMA acts as a container for one or more repair items and their associated documentation. This function serves as the central workflow around which all other features operate.

2.2.2. Customer Management

The application stores customer information including company name, contact name, address, telephone number, and email address. Customer information remains associated with each RMA to preserve historical accuracy and improve future searches.

2.2.3. Repair Item Management

Each RMA may contain multiple repair items. Individual repair items store machine identifiers, serial numbers, machine type, equipment descriptions, reported problems, repair notes, and received status, allowing multiple machines to be managed under a single authorization.

2.2.4. Shipping Management

The application records incoming and outgoing shipping information including carriers, tracking numbers, shipment dates, and receipt dates. This information provides complete shipment traceability throughout the repair process.

2.2.5. Status Management

Repair progress is monitored using configurable status values such as Created, Received, Diagnosing, Waiting on Parts, Repair Complete, Ready to Ship, and Closed. The current status provides an immediate indication of repair progress.

2.2.6. Status History

Every status change is permanently recorded with the previous status, new status, date, time, and optional notes. This historical log provides complete traceability for every RMA.

2.2.7. Searching

Users may search for records using numerous criteria including RMA number, customer name, company, machine ID, serial number, tracking number, machine type, or repair description. Search functionality enables rapid retrieval of historical records.

2.2.8. Filtering

Filtering allows users to display only records matching selected criteria, such as repair status or current search results. Filters simplify navigation when managing large numbers of RMAs.

2.2.9. Reporting

The reporting subsystem generates professional RMA reports suitable for internal documentation and customer communication. Reports summarize customer information, repair items, shipping information, and repair history.

2.2.10. Printing

Users may preview reports prior to printing, configure page settings, select printers, and produce professional multi-page printed reports with headers and footers.

2.2.11. Microsoft Excel Export

The application exports selected RMAs or filtered search results to Microsoft Excel (.xlsx) format while preserving structured tabular information for further analysis or archival.

2.2.12. PDF Export

Professional PDF reports may be generated for individual RMAs or report collections. PDF exports provide a portable document suitable for distribution, printing, and long-term storage.

2.2.13. Database Management

The application automatically initializes the SQLite database, manages connections, validates data, performs CRUD operations, and maintains database integrity throughout normal operation.

2.2.14. Backup and Recovery

Automatic and manual backup functions protect application data by creating recoverable database copies. Restore capabilities allow previous backups to be reinstated in the event of data loss or corruption. These features help ensure business continuity and minimize the risk of permanent data loss.

2.3. User Classes

This section identifies the primary categories of users who interact with RMA Tracker Version 2.1.1 and summarizes their roles within the organization. Although Version 2.1.1 does not currently implement role-based authentication or permissions, different users interact with the application in different ways based on their job responsibilities.

RMA Tracker was developed by Brennen Gabriel for Harp Enterprises to provide a centralized repair management solution that supports administrative personnel, management, and technical staff throughout the complete Return Merchandise Authorization (RMA) lifecycle. Understanding the intended user classes helps define how the application is expected to be used and provides a foundation for future role-based security enhancements.

The following subsections describe the major user classes supported by the current version of the application as well as user roles planned for future releases.

2.3.1. System Administrator

The System Administrator is responsible for installing, configuring, maintaining, and updating the application. Administrative responsibilities include database maintenance, backup verification, restoring database backups, troubleshooting software issues, and supporting future software deployments. Administrators require unrestricted access to all application functionality.

2.3.2. Office Personnel

Office Personnel represent the primary users of the application. They create new RMA records, update customer information, add repair items, record shipping information, modify repair statuses, search historical records, and generate customer reports. Their daily interaction with the software makes usability and efficiency especially important.

2.3.3. Managers and Supervisors

Managers and Supervisors primarily use the reporting, search, filtering, and monitoring capabilities of the application. They review repair workloads, monitor repair progress, evaluate historical trends, and generate reports used for operational planning and customer communication.

2.3.4. Information Technology Personnel

Information Technology personnel support the technical operation of RMA Tracker by installing software updates, maintaining workstation environments, validating backups, restoring databases when necessary, and assisting users with troubleshooting and system maintenance.

2.3.5. Future User Roles

Future commercial versions of RMA Tracker are expected to introduce configurable user roles with role-based permissions. Planned user classes include Read-Only Users, Shipping Personnel, Warehouse Staff, Quality Assurance Personnel, Customer Portal Users, and Executive Dashboard Users. These enhancements will improve security while allowing users to access only the functions required for their responsibilities.

2.4. Operating Environment

This section describes the hardware, software, and operating environment required to install, execute, and maintain RMA Tracker Version 2.1.1. Defining the operating environment ensures that the application functions consistently across supported platforms while providing acceptable performance, reliability, and compatibility.

RMA Tracker was developed by Brennen Gabriel for Harp Enterprises as a standalone desktop application intended for day-to-day repair management operations. The software has been designed to operate without requiring a dedicated server or continuous Internet connection, making it suitable for organizations that require a reliable local solution for managing Return Merchandise Authorization (RMA) records.

Although Version 2.1.1 focuses on local deployment, the application architecture supports future migration to enterprise environments. The following subsections describe the supported operating systems, software requirements, hardware requirements, storage needs, and optional peripheral devices that make up the intended operating environment.

2.4.1. Supported Operating Systems

The application is designed to operate on modern desktop operating systems. Version 2.1.1 supports Microsoft Windows 10 and Windows 11, macOS Monterey or later, and is expected to support Linux distributions in future releases. The software should execute consistently across supported platforms provided the required Java runtime environment is installed.

2.4.2. Software Requirements

RMA Tracker requires Java Development Kit (JDK) 24 or later, the JavaFX Runtime, and the embedded SQLite database engine. Report generation relies on Apache POI for Microsoft Excel exports and a compatible PDF generation library for PDF reports. No separate database server is required because SQLite is embedded within the application.

2.4.3. Hardware Requirements

The minimum recommended hardware consists of a dual-core processor, 4 GB of RAM, approximately 500 MB of available disk space, and a display resolution of at least 1366 × 768 pixels. For improved responsiveness, especially when managing larger databases and generating reports, a quad-core processor, 8 GB or more of RAM, solid-state storage (SSD), and a Full HD (1920 × 1080) display are recommended.

2.4.4. Storage and Network Environment

Application data, including the SQLite database, exported reports, and backup files, is stored on the local file system. Regular backups should be maintained on external or network storage to reduce the risk of data loss. An active Internet connection is not required for normal operation, although future versions may optionally integrate cloud synchronization and online shipment tracking services.

2.4.5. Optional Peripheral Devices

RMA Tracker can utilize several optional devices to enhance productivity, including laser or inkjet printers for printed reports, network printers, barcode scanners for rapid equipment identification, external USB storage devices for backups, and network-attached storage

(NAS) devices for centralized backup management. These peripherals are optional and are not required for the application to function.

2.5. Design and Implementation Constraints

This section identifies the primary technical, architectural, operational, and business constraints that influenced the design and implementation of RMA Tracker Version 2.1.1. Understanding these constraints provides context for many of the design decisions described throughout this Software Requirements Specification and establishes the boundaries within which the software operates.

RMA Tracker was developed by Brennen Gabriel for Harp Enterprises as a reliable, standalone desktop application. The project prioritized stability, maintainability, portability, and ease of deployment over unnecessary complexity. Consequently, several technology choices and design decisions were intentionally made to simplify installation, reduce external dependencies, and support long-term maintenance.

The following subsections describe the principal constraints affecting the current release and identify considerations that may influence future versions of the application.

2.5.1. Programming Language

RMA Tracker is implemented entirely in Java. Selecting Java provides platform independence, object-oriented design, a mature ecosystem of libraries, and long-term maintainability across multiple operating systems.

2.5.2. User Interface Framework

The graphical user interface is implemented using JavaFX. This framework provides a modern desktop interface, supports multiple operating systems, and allows a clear separation between presentation and application logic.

2.5.3. Database Technology

The application uses SQLite as its embedded relational database. SQLite eliminates the need for a dedicated database server, simplifies deployment, and stores all application data in a compact local database file while still providing transactional integrity.

2.5.4. Deployment Model

Version 2.1.1 is designed as a standalone desktop application. All processing occurs locally on the user's workstation, allowing the software to function independently of server infrastructure or cloud services.

2.5.5. Offline Operation

The application is designed to operate without a continuous Internet connection. All core functionality—including RMA management, searching, reporting, exporting, backups, and database operations—remains available while offline.

2.5.6. Security Constraints

The current version relies primarily on the host operating system for user authentication and file access permissions. Built-in user accounts, role-based authorization, and centralized authentication are planned for future releases.

2.5.7. Third-Party Libraries and Maintainability

External libraries are intentionally limited to well-supported components such as JavaFX, SQLite, Apache POI, and the PDF generation library. This minimizes maintenance risk, improves long-term compatibility, and supports future commercial distribution.

2.6. Assumptions and Dependencies

This section documents the assumptions and external dependencies that apply to RMA Tracker Version 2.1.1. These assumptions define the expected operating conditions under which the application has been designed, tested, and deployed. They also identify external software, hardware, and environmental factors that may influence the successful operation of the system.

RMA Tracker was developed by Brennen Gabriel for Harp Enterprises as a standalone desktop application intended to provide a reliable and maintainable repair management solution. Although the software has been designed to operate independently of network services and server infrastructure, it relies on several underlying assumptions regarding the user's computing environment, supporting software, and operational procedures.

The following assumptions and dependencies provide the foundation upon which the software requirements described throughout this document are based. Future versions of the application may introduce additional dependencies as enterprise features are implemented.

2.6.1. User Assumptions

The application assumes that users possess basic computer skills and have sufficient operating system permissions to execute the software, create reports, and access application files. Users are expected to follow established organizational procedures when creating and maintaining RMA records.

2.6.2. System Assumptions

RMA Tracker assumes that the host computer meets the minimum hardware and software requirements described in Section 2.4. Adequate disk space must be available for the SQLite database, exported reports, and backup files. The system clock should be correctly configured to ensure accurate timestamps.

2.6.3. Database Dependencies

The application depends on the embedded SQLite database engine for persistent data storage. Database files should not be modified by external applications while RMA Tracker is running, as doing so could affect data integrity or application stability.

2.6.4. File System and Backup Dependencies

The application assumes that the local file system is available and writable. Backup directories must remain accessible so that automatic and manual backups can be created successfully. Organizations are encouraged to store backup copies on external or network storage as part of their disaster recovery strategy.

2.6.5. Software Dependencies

Version 2.1.1 depends on the Java Runtime Environment, JavaFX, SQLite, Apache POI for Microsoft Excel exports, and the PDF generation library used by the application. These components must remain available and compatible for all supported functionality to operate correctly.

2.6.6. Operational Assumptions

The application is intended for use in an environment where repair records are maintained by authorized personnel. It assumes that users will perform routine backups, verify exported reports when required, and follow organizational policies for data protection and record retention.

2.6.7. Future Dependencies

Future releases may introduce additional dependencies including cloud-hosted databases, authentication services, shipping carrier APIs, email services, REST APIs, and customer web portals. The modular architecture of Version 2.1.1 has been designed to support these enhancements without requiring significant changes to the existing system.

3. Functional Requirements

This section defines the functional requirements for RMA Tracker Version 2.1.1. Each functional requirement identifies the expected behavior of the system using formal requirement statements. The requirements describe the functionality implemented in Version 2.1.1 and provide a foundation for future verification and testing.

3.1. FR-1 Create RMA

3.1.1. Description

FR-1 Create RMA provides the functionality required to support the RMA lifecycle within the application.

3.1.2. Functional Requirements

- **FR-1.1** The system shall allow a user to create a new RMA record.
- **FR-1.2** The system shall assign a unique RMA number.
- **FR-1.3** The system shall save customer information.
- **FR-1.4** The system shall save shipping information.
- **FR-1.5** The system shall display the new RMA in the main table.

3.1.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.2. FR-2 Edit RMA

3.2.1. Description

FR-2 Edit RMA provides the functionality required to support the RMA lifecycle within the application.

3.2.2. Functional Requirements

- **FR-2.1** The system shall allow editing of existing RMAs.
- **FR-2.2** The system shall validate modified fields before saving.
- **FR-2.3** The system shall update the database immediately after saving.
- **FR-2.4** The system shall refresh the displayed information.

3.2.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.3. FR-3 Delete RMA

3.3.1. Description

FR-3 Delete RMA provides the functionality required to support the RMA lifecycle within the application.

3.3.2. Functional Requirements

- **FR-3.1** The system shall allow deletion of an RMA after confirmation.
- **FR-3.2** The system shall remove associated repair items.
- **FR-3.3** The system shall refresh the RMA list.

3.3.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.4. FR-4 Search

3.4.1. Description

FR-4 Search provides the functionality required to support the RMA lifecycle within the application.

3.4.2. Functional Requirements

- **FR-4.1** The system shall search by RMA number.
- **FR-4.2** The system shall search by customer.
- **FR-4.3** The system shall search by serial number.
- **FR-4.4** The system shall update results as search criteria change.

3.4.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.5. FR-5 Status Filtering

3.5.1. Description

FR-5 Status Filtering provides the functionality required to support the RMA lifecycle within the application.

3.5.2. Functional Requirements

- **FR-5.1** The system shall filter RMAs by status.
- **FR-5.2** The system shall display only matching records.
- **FR-5.3** The system shall clear filters on request.

3.5.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.6. FR-6 Repair Item Management

3.6.1. Description

FR-6 Repair Item Management provides the functionality required to support the RMA lifecycle within the application.

3.6.2. Functional Requirements

- **FR-6.1** The system shall add repair items.
- **FR-6.2** The system shall edit repair items.
- **FR-6.3** The system shall delete repair items.
- **FR-6.4** The system shall support multiple repair items per RMA.

3.6.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.7. FR-7 Shipping

3.7.1. Description

FR-7 Shipping provides the functionality required to support the RMA lifecycle within the application.

3.7.2. Functional Requirements

- **FR-7.1** The system shall store incoming tracking numbers.
- **FR-7.2** The system shall store outgoing tracking numbers.
- **FR-7.3** The system shall allow modification of shipping information.

3.7.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.8. FR-8 Status History

3.8.1. Description

FR-8 Status History provides the functionality required to support the RMA lifecycle within the application.

3.8.2. Functional Requirements

- **FR-8.1** The system shall record status changes.
- **FR-8.2** The system shall maintain chronological history.
- **FR-8.3** The system shall display status history to the user.

3.8.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.9. FR-9 Report Preview

3.9.1. Description

FR-9 Report Preview provides the functionality required to support the RMA lifecycle within the application.

3.9.2. Functional Requirements

- **FR-9.1** The system shall generate a preview before printing.
- **FR-9.2** The preview shall include customer and repair information.

3.9.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.10. FR-10 Printing

3.10.1. Description

FR-10 Printing provides the functionality required to support the RMA lifecycle within the application.

3.10.2. Functional Requirements

- **FR-10.1** The system shall print the selected RMA.
- **FR-10.2** The system shall print formatted reports.

3.10.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.11. FR-11 Excel Export

3.11.1. Description

FR-11 Excel Export provides the functionality required to support the RMA lifecycle within the application.

3.11.2. Functional Requirements

- **FR-11.1** The system shall export filtered RMAs to Excel.
- **FR-11.2** Each repair item shall occupy one row.
- **FR-11.3** The export shall preserve current filters.

3.11.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.12. FR-12 PDF Export

3.12.1. Description

FR-12 PDF Export provides the functionality required to support the RMA lifecycle within the application.

3.12.2. Functional Requirements

- **FR-12.1** The system shall export selected RMAs to PDF.
- **FR-12.2** The system shall export filtered results to PDF.

- **FR-12.3** The generated PDF shall preserve report formatting.

3.12.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.13. FR-13 Backup

3.13.1. Description

FR-13 Backup provides the functionality required to support the RMA lifecycle within the application.

3.13.2. Functional Requirements

- **FR-13.1** The system shall create manual backups.
- **FR-13.2** The system shall support automatic backups.
- **FR-13.3** The system shall notify the user if backup fails.

3.13.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.14. FR-14 Restore

3.14.1. Description

FR-14 Restore provides the functionality required to support the RMA lifecycle within the application.

3.14.2. Functional Requirements

- **FR-14.1** The system shall restore the database from a backup.
- **FR-14.2** The system shall validate the selected backup before restoration.

3.14.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.15. FR-15 Database Initialization

3.15.1. Description

FR-15 Database Initialization provides the functionality required to support the RMA lifecycle within the application.

3.15.2. Functional Requirements

- **FR-15.1** The system shall initialize the SQLite database during startup.
- **FR-15.2** The system shall create missing tables automatically.
- **FR-15.3** The system shall connect to an existing database when available.

3.15.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

3.16. FR-16 Validation

3.16.1. Description

FR-16 Validation provides the functionality required to support the RMA lifecycle within the application.

3.16.2. Functional Requirements

- **FR-16.1** The system shall validate required fields.
- **FR-16.2** The system shall prevent invalid data from being saved.
- **FR-16.3** The system shall display descriptive validation messages.

3.16.3. Expected Outcome

Successful completion of this function updates the application interface and persists all valid changes to the database where applicable.

4. External Interface Requirements

External Interface Requirements define how users interact with RMA Tracker Version 2.1.1 and how information is presented throughout the application. The application provides a desktop graphical user interface built with JavaFX and is designed to provide consistent, efficient, and intuitive workflows for creating, tracking, updating, searching, reporting, and exporting Return Merchandise Authorizations (RMAs). The following sections describe the user interface, menus, dialogs, tables, and keyboard shortcuts implemented in Version 2.1.1.

4.1. User Interface

The application uses a desktop JavaFX interface organized around a primary main window. The interface emphasizes clarity, minimal navigation, and rapid access to frequently used functions.

General requirements:

- Consistent window layout across all screens.
- Standard toolbar and menu bar.
- Search and filtering controls remain easily accessible.
- Status bar displays operation feedback.
- Responsive resizing of tables and controls.
- Color-coded status values where appropriate.
- Validation messages displayed immediately when invalid data is entered.
- Modal dialogs used for confirmation and critical operations.

Primary screens include:

- Main Dashboard
- Create/Edit RMA window
- Repair Item management
- Shipping information
- Report Preview
- Backup/Restore utilities
- Settings dialogs
- Export and print interfaces

UI Requirements:

- UI-1 The application shall present a single primary application window.
- UI-2 All primary functions shall be reachable within three user interactions.
- UI-3 Required fields shall be visually identified.
- UI-4 Invalid entries shall display descriptive validation messages.
- UI-5 Long-running operations shall provide visual feedback.
- UI-6 Table layouts shall remain readable after window resizing.

4.2. Menus

The menu system provides access to every major function.

Typical menus include:

- File
- New RMA
- Open
- Backup Database
- Restore Database
- Print
- Export PDF
- Export Excel
- Exit

- Edit
- Edit Selected RMA
- Delete Selected RMA
- Preferences

- View
- Refresh
- Filter by Status
- Report Preview

- Tools
- Machine Types
- Settings
- Database Maintenance

- Help
- About
- Documentation

Menu Requirements

- MEN-1 Menu labels shall remain consistent.
- MEN-2 Disabled commands shall be unavailable when no record is selected.
- MEN-3 Confirmation shall be required before destructive actions.
- MEN-4 Menu commands shall invoke the same functionality as toolbar buttons.

4.3. Dialogs

Dialogs provide focused interaction for creating records, editing data, confirmations, and notifications.

Dialogs include:

- Create RMA
- Edit RMA
- Delete Confirmation
- Backup Complete
- Restore Confirmation
- Export Options
- Error Notifications
- About dialog

Dialog Requirements

- DLG-1 Dialogs shall prevent invalid submissions.
- DLG-2 Confirmation dialogs shall be displayed before permanent deletion.
- DLG-3 Error dialogs shall include a meaningful description.
- DLG-4 Success dialogs shall confirm completed operations.
- DLG-5 Modal dialogs shall prevent conflicting actions while open.

4.4. Tables

Tables present RMAs, repair items, and search results.

Displayed information may include:

- RMA Number
- Customer
- Machine Type
- Serial Number
- Status
- Date Received
- Date Shipped

- Technician
- Tracking Numbers

Table Requirements

- TBL-1 Users shall be able to sort columns.
- TBL-2 Users shall be able to resize columns.
- TBL-3 Search filters shall update displayed rows.
- TBL-4 Status filters shall display only matching records.
- TBL-5 Selected rows shall be visually highlighted.
- TBL-6 Large datasets shall remain responsive.

4.5. Keyboard Shortcuts

Keyboard shortcuts improve efficiency for experienced users.

Suggested shortcuts:

- Ctrl/Cmd+N – Create New RMA
- Ctrl/Cmd+S – Save
- Ctrl/Cmd+F – Search
- Ctrl/Cmd+P – Print
- Ctrl/Cmd+E – Export
- Delete – Delete selected RMA
- F5 – Refresh
- Esc – Close active dialog
- Enter – Confirm dialog

Shortcut Requirements:

- KEY-1 Shortcuts shall invoke the same commands as menus.
- KEY-2 Platform conventions shall be respected (Ctrl on Windows/Linux, Command on macOS).
- KEY-3 Shortcuts shall be disabled when inappropriate for the current context.
- KEY-4 Users shall retain full fun

5. Nonfunctional Requirements

This section defines the non-functional requirements for RMA Tracker Version 2.1.1. These requirements describe the quality attributes, constraints, and operational characteristics of the application. Unlike functional requirements, they specify how the system performs rather than what functions it performs.

5.1. Performance

- The application shall launch within 5 seconds on supported hardware.
- Database searches shall return results within 2 seconds for databases containing up to 100,000 RMA records.
- Creating, editing, and saving RMAs shall normally complete within 1 second.
- Report generation shall complete within 10 seconds for reports containing up to 5,000 repair items.
- PDF and Excel exports shall complete within 15 seconds under normal operating conditions.
- The application shall support continuous operation during a standard 8-hour workday without degradation.

5.2. Reliability

- The system shall maintain database integrity during unexpected shutdowns.
- Automatic database backups shall be created at application exit.
- Backup restoration shall preserve all valid records.
- The application shall gracefully handle invalid user input without crashing.
- Unexpected errors shall be logged and displayed using user-friendly error messages.

5.3. Availability

- The application is intended for desktop use and shall be available whenever the host workstation is operational.
- Internet connectivity is not required for normal operation.

5.4. Security

- Current Version 2.1.1 is designed for trusted internal environments.
- The application shall validate all user input.
- SQLite database access shall be limited by operating system file permissions.
- Future versions may include user authentication, role-based authorization, password management, encryption, and audit logging.

5.5. Maintainability

- The application is developed using Java 21, JavaFX, Gradle, SQLite, and MVC architecture.
- Source code shall follow consistent naming conventions and Javadoc documentation.
- Business logic, UI, and data access layers shall remain separated.
- New modules should be added with minimal impact to existing functionality.

5.6. Portability

- The application shall run on supported versions of macOS and Windows with an installed Java Runtime where required or packaged runtime.
- Database files shall remain portable between supported operating systems.

5.7. Usability

- The interface shall remain consistent across all windows.
- Common actions shall require minimal training.
- Validation messages shall clearly explain corrective actions.
- Tables shall support sorting, filtering, and resizing where implemented.

5.8. Scalability

- The database shall support at least 100,000 RMAs and multiple repair items per RMA without significant performance degradation.
- Future database engines may replace SQLite with minimal application changes.

5.9. Compatibility

- The application shall support PDF export, Microsoft Excel (.xlsx) export, standard printers, and operating-system file dialogs.

5.10. Backup and Recovery

- Automatic backups shall be stored in the application backup directory.
- Manual backups shall be supported.
- Restore operations shall verify backup integrity before replacing the active database.

5.11. Logging and Diagnostics

- Application startup, shutdown, backup operations, and major system errors shall be logged.
- Diagnostic information shall assist troubleshooting while avoiding disclosure of sensitive information.

5.12. Compliance and Standards

- The SRS follows IEEE 29148 guidance.
- Source code shall follow Java coding conventions.
- Date formats, filenames, and exported documents shall remain consistent throughout the application.

5.13. Design Goals for Future Releases

- Role-based security and authentication.
- Automatic application updates.
- Cloud synchronization and centralized database support.
- REST API integration with third-party ERP systems.
- Enhanced audit trails and reporting analytics.

6. Data Requirements

This section defines the data requirements for RMA Tracker Version 2.1.1. It describes the logical data model, data entities, validation rules, integrity constraints, storage requirements, backup and recovery expectations, retention policies, and future scalability. These requirements ensure that all information associated with the Return Merchandise Authorization (RMA) lifecycle is stored accurately, securely, and consistently.

6.1. Database Schema

RMA Tracker uses a relational SQLite database to store application data. Core entities include RMAs, Customers, Repair Items, Shipping Records, Status History, Machine Types, Manufacturers, and Backup Metadata. Relationships are normalized to reduce redundancy while maintaining efficient reporting and searching.

6.2. Data Entities

Primary entities include:

- RMA
- Customer
- Repair Item
- Manufacturer
- Machine Type
- Shipping Record
- Status History
- User Preferences
- Application Settings
- Backup Records

6.3. Data Dictionary

Each entity contains uniquely defined fields with appropriate data types, validation rules, default values, and business constraints. Primary keys are unique integers, foreign keys enforce referential integrity, and date/time values use ISO-8601 formatting where applicable.

6.4. Data Validation

The application shall validate required fields before saving records. Validation includes mandatory customer information, valid serial numbers when required, acceptable status values, valid shipping dates, duplicate detection where applicable, and numeric validation for quantities.

6.5. Data Integrity

The system shall preserve referential integrity between related records. Cascading updates shall be controlled to prevent accidental data loss. Transactions shall complete atomically so partially written records cannot occur.

6.6. Data Storage

Application data shall be stored in an SQLite database located within the user's application data directory. Backup copies shall be stored separately from the active database to support recovery.

6.7. Backup and Recovery

The application shall support automatic and manual database backups. Recovery shall allow restoration from previous backup files while preserving database consistency. Backup failures shall be logged and reported to the user.

6.8. Data Retention

RMA records shall remain available until intentionally deleted by authorized users. Historical status information shall be retained to support auditing and reporting.

6.9. Reporting Data

Stored data shall support report generation, printing, PDF export, Excel export, filtering, sorting, and future dashboard functionality without modifying the original records.

6.10. Performance Requirements

The database shall support rapid searching, filtering, and report generation for thousands of RMA records with acceptable desktop performance.

6.11. Security Requirements

The application shall protect stored information through operating-system file permissions. Future releases may include encryption, user authentication, role-based authorization, and audit logging.

6.12. Future Data Enhancements

Future versions may introduce multi-user database servers, cloud synchronization, API integration with shipping carriers, automatic tracking updates, barcode support, customer portals, and advanced analytics.

7. Business Rules

This section defines the business rules governing the operation of RMA Tracker Version 2.1.1. Business rules establish the policies, constraints, validations, and operational standards that ensure all Return Merchandise Authorization (RMA) records are processed consistently throughout their lifecycle. These rules support data integrity, reporting accuracy, auditability, and standardized workflows across Harp Enterprises.

7.1. BR-1 RMA Number Generation

Each RMA shall have a unique identifier. Duplicate RMA numbers shall not be permitted.

7.2. BR-2 Required Information

An RMA shall not be saved unless all required customer and repair information has been entered.

7.3. BR-3 Repair Item Association

Every repair item shall belong to exactly one RMA. An RMA may contain one or more repair items.

7.4. BR-4 Status Management

Status values shall follow the defined workflow. Status changes shall be recorded in chronological order.

7.5. BR-5 Shipping Validation

Shipping information shall not be marked complete until required shipment data has been entered.

7.6. BR-6 Data Integrity

Deleting an RMA shall also remove associated child records only through controlled application logic to prevent orphaned records.

7.7. BR-7 Search and Filtering

Searches shall return records matching user-defined criteria without modifying stored data.

7.8. BR-8 Report Generation

Generated reports shall accurately reflect the database contents at the time the report is created.

7.9. BR-9 Export Rules

Exported PDF and Excel files shall preserve displayed data, formatting, and selected filters.

7.10. BR-10 Backup and Recovery

Database backups shall be created through the application's backup functions. Restores shall replace the active database only after user confirmation.

7.11. BR-11 Audit History

Status history shall remain associated with its originating RMA and shall not be editable once recorded.

7.12. BR-12 Future Business Rules

Future versions may introduce configurable workflows, approval requirements, user permissions, customer-specific rules, SLA tracking, automated notifications, barcode processing, and ERP integration without affecting existing business rules.

8. Use Cases

Use cases describe the interactions between users and RMA Tracker to accomplish business tasks. They define the expected system behavior, normal workflows, alternate flows, validation rules, and postconditions.

8.1.1. UC-1 Create RMA

Administrative user creates a new RMA by entering customer, machine, serial number, problem description and related repair items. System validates required fields, generates a unique RMA number and stores the record.

8.1.2. UC-2 Search RMAs

User searches by RMA number, customer, serial number, machine type, or status. Matching records are displayed with sorting and filtering.

8.1.3. UC-3 Update RMA

Authorized user edits customer information, repair notes, shipping information or status. All changes are saved immediately.

8.1.4. UC-4 Manage Repair Items

User adds, edits or removes repair items associated with an RMA.

8.1.5. UC-5 Update Status

User changes the repair status. Status history is recorded chronologically.

8.1.6. UC-6 Shipping

User records inbound/outbound shipping information, tracking numbers and shipment dates.

8.1.7. UC-7 Reports

User previews, prints or exports individual RMAs or filtered lists to PDF or Excel.

8.1.8. UC-8 Backup and Restore

Administrator creates manual backups, restores previous backups and verifies database integrity.

8.1.9. Use Case Business Rules

- Each RMA shall have a unique identifier.
- Required fields shall be validated before saving.

- Deleted RMAs shall require confirmation.
- Status history shall not be modified after creation.
- Exported reports shall match displayed data.

9. Future Enhancements

The following enhancements are outside the scope of Version 2.1.1 but represent the planned product roadmap.

9.1.1. Machine Type Controller

Create a centralized machine type management module supporting configurable machine categories, validation rules, default settings and future inventory integration.

9.1.2. Automatic Tracking Updates

Integrate shipping carrier APIs (UPS, FedEx, USPS, etc.) to automatically retrieve shipment status and delivery information from tracking numbers.

9.1.3. Cloud Sync

Support encrypted synchronization between multiple installations using a cloud-hosted database with conflict detection and offline synchronization.

9.1.4. User Accounts

Implement authentication, role-based permissions, password policies, audit logging and Active Directory/LDAP integration.

9.1.5. Dashboard

Provide management dashboards displaying repair workload, turnaround time, technician productivity, overdue repairs and KPIs using interactive charts.

10.Acceptance Criteria

The following criteria define the minimum requirements for successful acceptance of RMA Tracker Version 2.1.1.

- The application installs and launches successfully on supported operating systems.
- Users can create, edit, search and delete RMAs without data corruption.
- Repair items can be added, modified and removed correctly.
- Status updates are saved accurately and reflected in searches and reports.
- Shipping information and tracking fields are stored correctly.
- PDF and Excel exports generate successfully.
- Automatic and manual database backups complete successfully.
- Restore operations recover valid database backups.
- Application remains stable during extended operation without crashes.
- Performance remains acceptable with large RMA datasets.
- Validation prevents incomplete or invalid required data.
- All regression and functional tests pass before release.

11.Glossary

The following glossary defines terminology used throughout the RMA Tracker Version 2.1.1 Software Requirements Specification. These terms provide a common understanding of business, technical, and application-specific concepts referenced throughout this document.

Term	Definition
Application Settings	Configuration values that control application behavior, default options, backup locations, and user preferences.
Archive	The process of preserving historical RMA records or software versions for future reference.
Backup	A copy of the SQLite database created to recover from data loss or corruption.
Business Rules	Policies governing how RMA Tracker validates and processes data.
CRUD	Create, Read, Update, and Delete operations performed on database records.
Customer	The organization or individual requesting repair services.
Database	The SQLite database used to store all application information.
Database Restore	Replacing the active database with a previously created backup.
Export	Saving application data to another format such as PDF or Microsoft Excel.
Filter	Criteria used to display only records matching selected conditions.
GUI	Graphical User Interface used to interact with the application.
History Log	Chronological record of status changes and significant events.
JavaFX	Framework used to build the desktop user interface.
Machine Type	Classification assigned to a repair item, such as Scanner or ADA device.
PDF Report	Portable Document Format report generated by the application.
Print Preview	On-screen display of a report before printing.
Repair Item	An individual machine or component associated with an RMA.
Repair Status	Current stage of the repair process.
RMA	Return Merchandise Authorization identifying a repair transaction.
RMA Record	Complete collection of information associated with one RMA.

Search Criteria	Values used to locate records within the application.
SQLite	Embedded relational database engine used by RMA Tracker.
Status History	Permanent chronological record of all status changes.
Tracking Number	Shipment identifier assigned by a shipping carrier.
User Interface	Visual components through which users interact with RMA Tracker.
Validation	Verification that entered information satisfies business rules before saving.
Version Number	Release identifier assigned to a software build.
Work Order	Documentation describing repair activities.
XLSX Export	Microsoft Excel workbook generated by the application.

11.1. Glossary Usage

This glossary supplements Section 1.4 (Definitions, Acronyms, and Abbreviations). Section 1.4 provides concise definitions and abbreviations required early in the document, while this glossary serves as a comprehensive reference for readers throughout the SRS.

12. Appendices

This section contains supplemental reference material that supports the Software Requirements Specification for RMA Tracker Version 2.1.1. The appendices provide technical references, visual documentation, configuration details, and supporting information that assist developers, testers, system administrators, and future maintainers. While not required to understand the system requirements, these materials provide valuable context for implementation, deployment, maintenance, and future enhancements.

12.1. Screenshots

- 12.1.1. Main Dashboard
- 12.1.2. Create RMA Window
- 12.1.3. Edit RMA Window
- 12.1.4. Search and Filter Interface
- 12.1.5. Repair Item Management
- 12.1.6. Shipping Information
- 12.1.7. Status History
- 12.1.8. Report Preview
- 12.1.9. Backup and Restore Windows
- 12.1.10. Settings Window

12.2. Database Schema

- 12.2.1. Database Overview
- 12.2.2. Entity Relationship Diagram (ERD)
- 12.2.3. Database Tables
- 12.2.4. Table Relationships
- 12.2.5. Primary Keys and Foreign Keys
- 12.2.6. Indexes and Constraints
- 12.2.7. Data Dictionary

12.3. Folder Structure

- 12.3.1. Project Directory Structure
- 12.3.2. Source Code Organization
- 12.3.3. Resource Files
- 12.3.4. Database Files
- 12.3.5. Backup Directory
- 12.3.6. Build Output
- 12.3.7. Documentation Folder

12.4. Error Messages

- 12.4.1. Validation Errors
- 12.4.2. Database Errors
- 12.4.3. Backup Errors
- 12.4.4. Restore Errors
- 12.4.5. Import and Export Errors
- 12.4.6. Printing Errors
- 12.4.7. File System Errors
- 12.4.8. Unexpected Exceptions

12.5. Version History

- 12.5.1. Release Timeline
- 12.5.2. Version 1.x Changes
- 12.5.3. Version 2.0 Changes
- 12.5.4. Version 2.1 Changes
- 12.5.5. Version 2.1.1 Changes
- 12.5.6. Planned Future Releases

12.6. Configuration Files

- 12.6.1. Application Configuration
- 12.6.2. Database Configuration
- 12.6.3. Backup Configuration
- 12.6.4. Build Configuration (Gradle)
- 12.6.5. Logging Configuration

12.7. Keyboard Shortcuts

- 12.7.1. Navigation Shortcuts
- 12.7.2. Record Management Shortcuts
- 12.7.3. Search Shortcuts
- 12.7.4. Reporting Shortcuts
- 12.7.5. Backup and Maintenance Shortcuts

12.8. Third-Party Libraries

- 12.8.1. JavaFX
- 12.8.2. SQLite JDBC
- 12.8.3. Apache POI
- 12.8.4. PDF Generation Library
- 12.8.5. Gradle Build System

12.9. Supporting Documents

- 12.9.1. Volume II – Technical Design and Architecture
- 12.9.2. Volume III – User Manual
- 12.9.3. Volume IV – Installation, Deployment, and Maintenance Guide
- 12.9.4. Release Notes
- 12.9.5. Project Repository Information